

Matplotlib Guide

Matplotlib is a powerful library for creating static, animated, and interactive visualizations in Python. It provides a wide range of plotting functions and customization options to create various types of plots, such as line plots, scatter plots, bar charts, histograms, and more.

1. Installation & Setup

To install matplotlib via terminal, you can use:

```
pip install matplotlib
```

```
In [2]: import matplotlib.pyplot as plt

import numpy as np
import pandas as pd
```

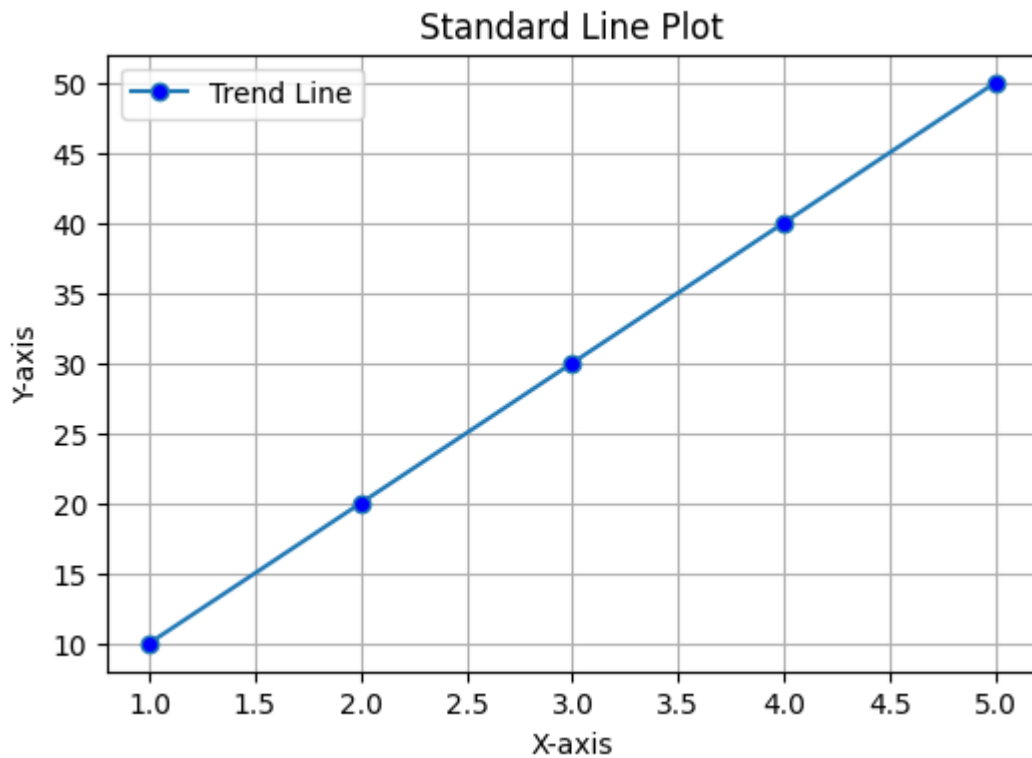
2. Core Plot Types

Line Plot (`plt.plot`)

Used to map continuous changes over an interval by interpolating lines between points.

```
In [3]: x_line = [1, 2, 3, 4, 5]
y_line = [10, 20, 30, 40, 50]

plt.figure(figsize=(6, 4))
plt.plot(x_line, y_line, marker='o', markerfacecolor='blue', linestyle='--')
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Standard Line Plot")
plt.legend()
plt.grid(True)
plt.show()
```

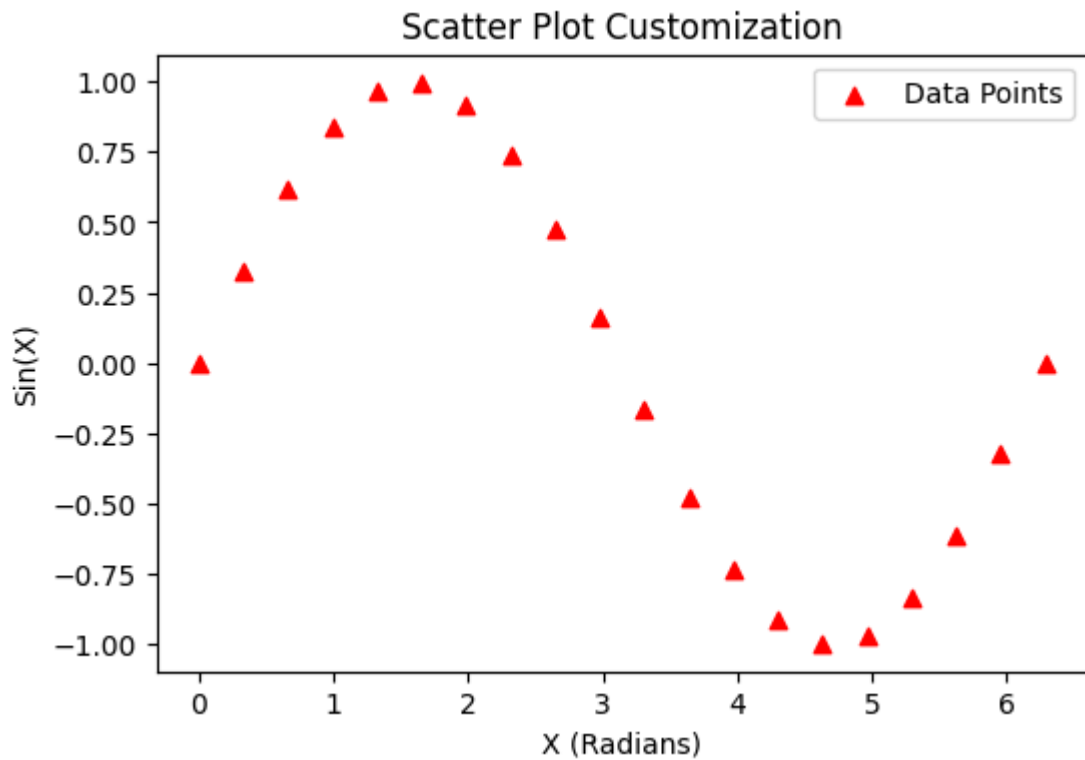


Scatter Plot (`plt.scatter`)

Plots points individually without connecting lines. Ideal for analyzing correlations between numeric arrays.

```
In [4]: x_scatter = np.linspace(0, 2 * np.pi, 20)
        y_scatter = np.sin(x_scatter)

        plt.figure(figsize=(6, 4))
        plt.scatter(x_scatter, y_scatter, color='red', marker='^', label='Data Po
        plt.xlabel("X (Radians)")
        plt.ylabel("Sin(X)")
        plt.title("Scatter Plot Customization")
        plt.legend()
        plt.show()
```

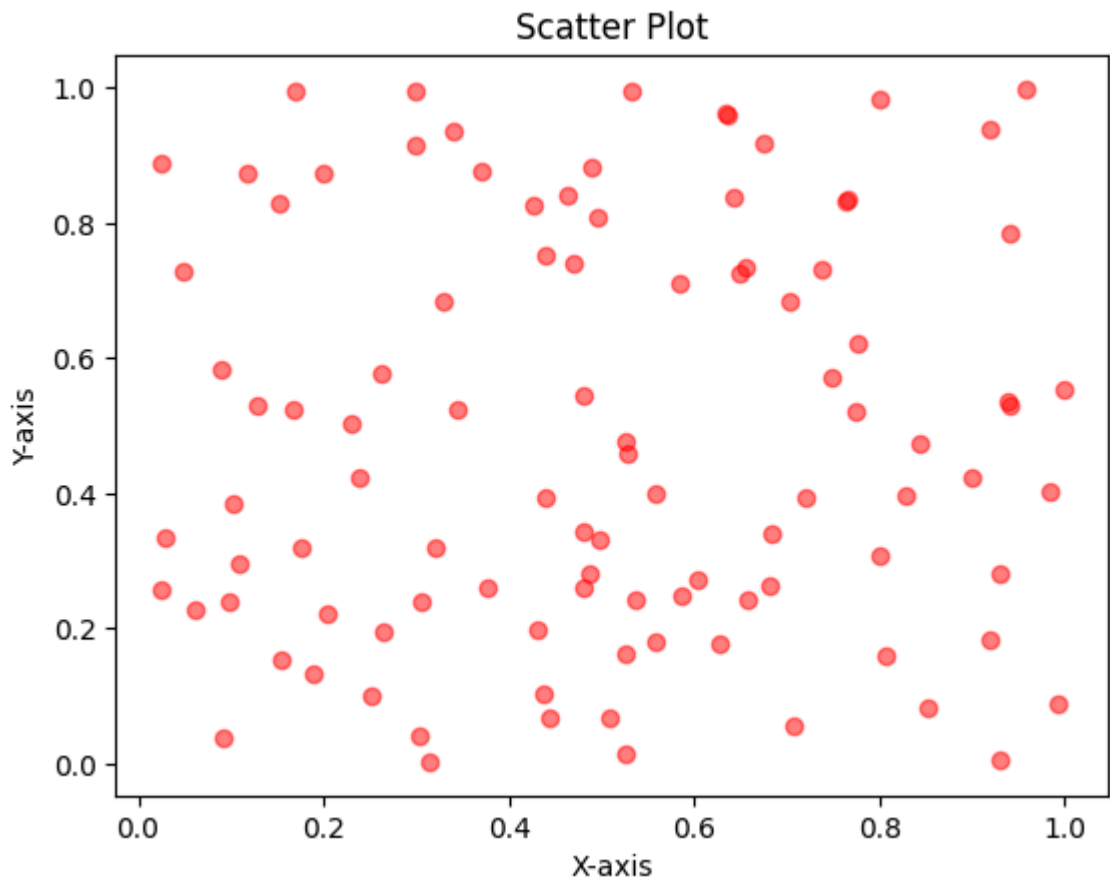


Random Scatter Plot (`plt.scatter`)

Plots randomly generated points individually without connecting lines. Ideal for testing plot layouts, visualizing noise distributions, or simulating data scatter.

```
In [5]: x_scatter = np.random.rand(100)
        y_scatter = np.random.rand(100)

        plt.scatter(x_scatter, y_scatter, color='red', alpha=0.5)
        plt.xlabel("X-axis")
        plt.ylabel("Y-axis")
        plt.title("Scatter Plot")
        plt.show()
```

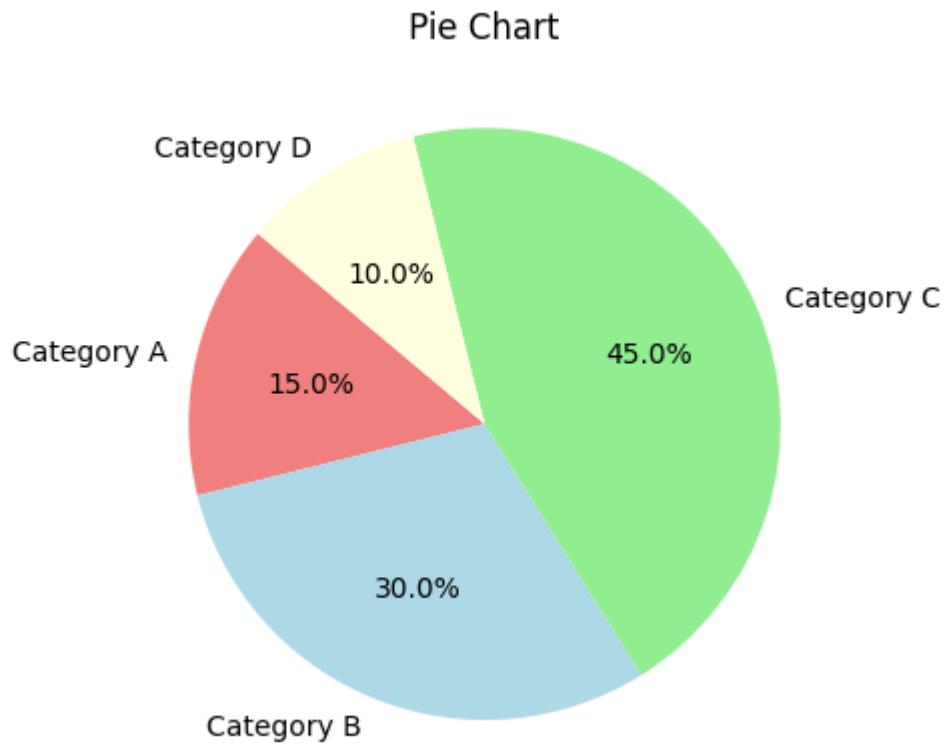


Pie Chart (plt.pie)

Plots data as slices of a circle to illustrate numerical proportions. Ideal for comparing the relative percentage or breakdown of categorical parts that make up a distinct whole (summing up to 100%)

```
In [6]: labels = ['Category A', 'Category B', 'Category C', 'Category D']
        sizes = [15, 30, 45, 10]

        plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=['lightcoral', 'lightblue', 'lightgreen', 'lightyellow'])
        plt.title("Pie Chart")
        plt.show()
```

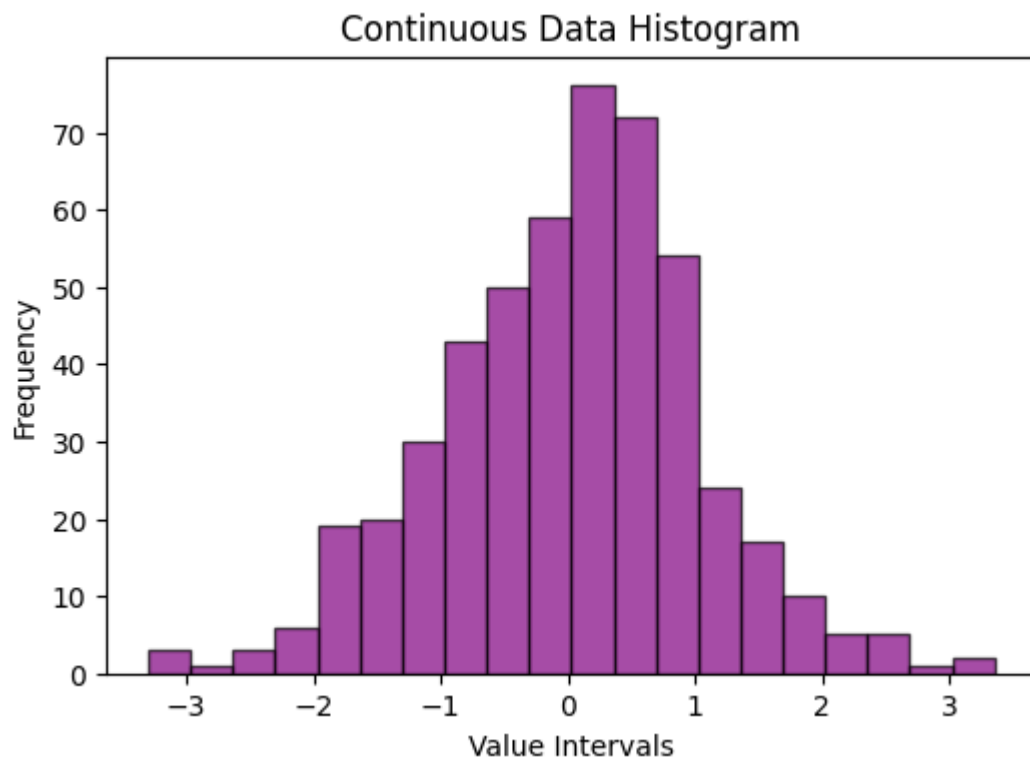


Histogram (`plt.hist`)

Groups continuous numerical data into frequency bins to show distribution shapes.

```
In [7]: samples = np.random.normal(loc=0.0, scale=1.0, size=500)

plt.figure(figsize=(6, 4))
plt.hist(samples, bins=20, color='purple', edgecolor='black', alpha=0.7)
plt.xlabel("Value Intervals")
plt.ylabel("Frequency")
plt.title("Continuous Data Histogram")
plt.show()
```

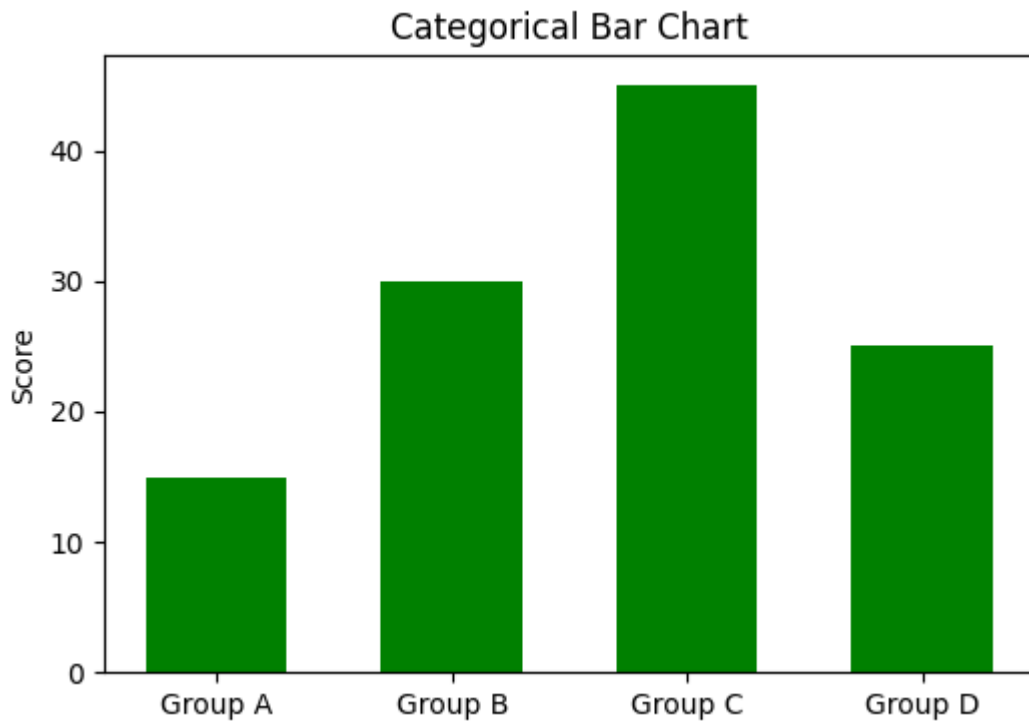


Bar Chart (`plt.bar`)

Compares individual quantitative metrics across separate categorical strings.

```
In [8]: categories = ['Group A', 'Group B', 'Group C', 'Group D']
performances = [15, 30, 45, 25]

plt.figure(figsize=(6, 4))
plt.bar(categories, performances, color='green', width=0.6)
plt.ylabel("Score")
plt.title("Categorical Bar Chart")
plt.show()
```



Error Bar (`plt.errorbar`)

Plots points with associated experimental horizontal and vertical uncertainty markers, without drawing a connecting line. Ideal for representing data from physical measurements with explicit error bars and custom cap widths.

```
In [9]: x3 = [1, 2, 3, 4, 5]
y3 = [2.1, 3.9, 6.2, 8.1, 10.2]
error_x = [0.1, 0.15, 0.1, 0.2, 0.12]
error_y = [0.2, 0.3, 0.25, 0.4, 0.35]

plt.figure(figsize=(6, 4))
plt.errorbar(x3, y3, xerr=error_x, yerr=error_y, c='purple', capsize=5,
plt.xlabel("Independent Variable (X)")
plt.ylabel("Measured Value (Y)")
plt.title("Experimental Data with Error Bars")
plt.grid(True)
plt.show()
```



3. Subplots

```
In [10]: x = np.linspace(0, 2 * np.pi, 100)
y_sin = np.sin(x)
y_cos = np.cos(x)

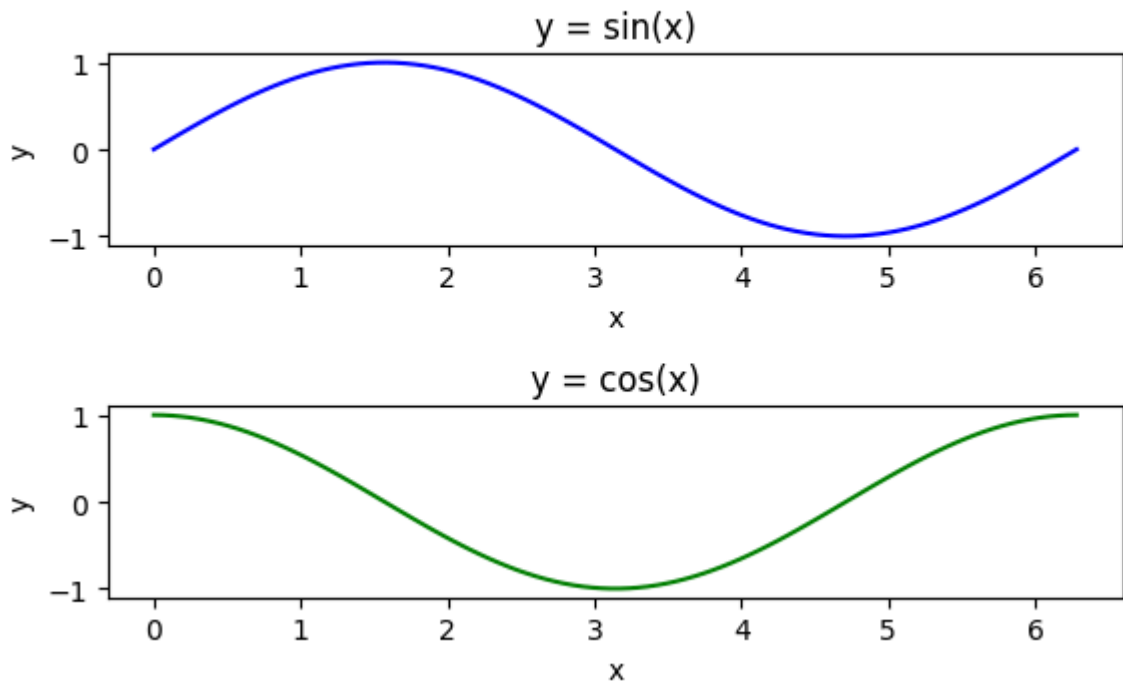
fig, axs = plt.subplots(2, 1, figsize=(6, 4))

# sin(x) plot
axs[0].plot(x, y_sin, 'b')
axs[0].set_title('y = sin(x)')
axs[0].set_xlabel('x')
axs[0].set_ylabel('y')

# cos(x) plot
axs[1].plot(x, y_cos, 'g')
axs[1].set_title('y = cos(x)')
axs[1].set_xlabel('x')
axs[1].set_ylabel('y')

plt.suptitle('Multiple Subplots of sin(x) and cos(x)')
plt.tight_layout()
plt.show()
```

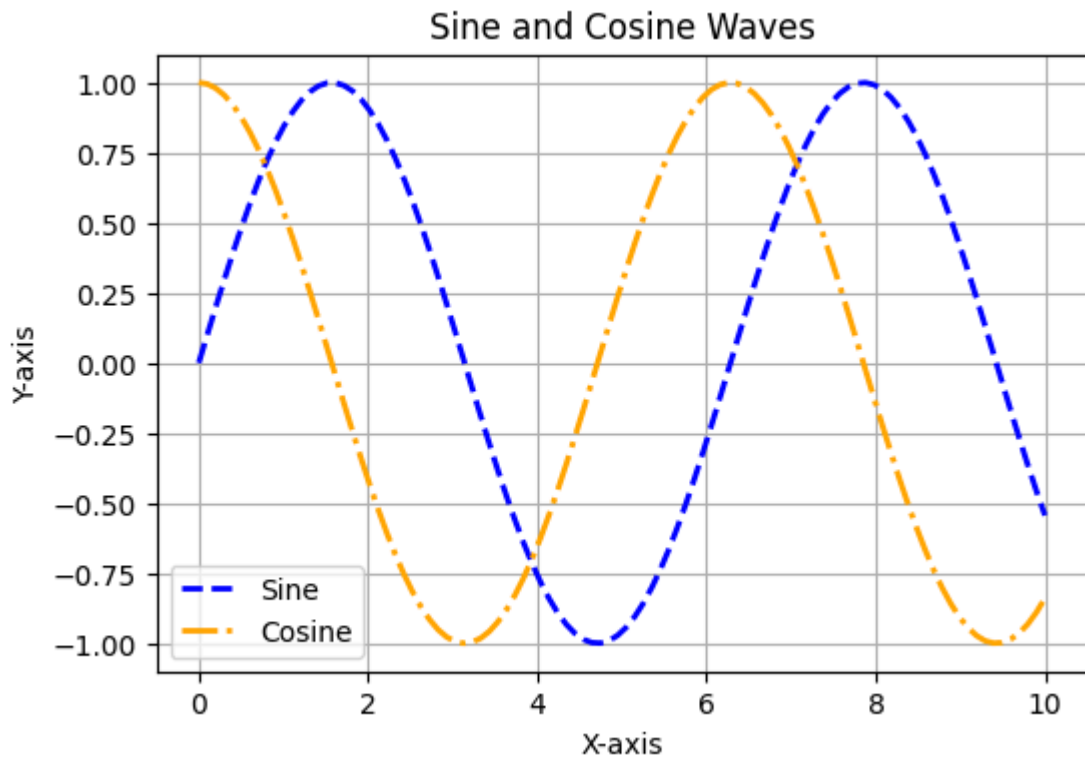
Multiple Subplots of $\sin(x)$ and $\cos(x)$



4. Different lines to the same plot

```
In [11]: x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.figure(figsize=(6, 4))
plt.plot(x, y1, label='Sine', color='blue', linestyle='--', linewidth=2)
plt.plot(x, y2, label='Cosine', color='orange', linestyle='-.', linewidth=2)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Sine and Cosine Waves")
plt.legend()
plt.grid(True)
plt.show()
```

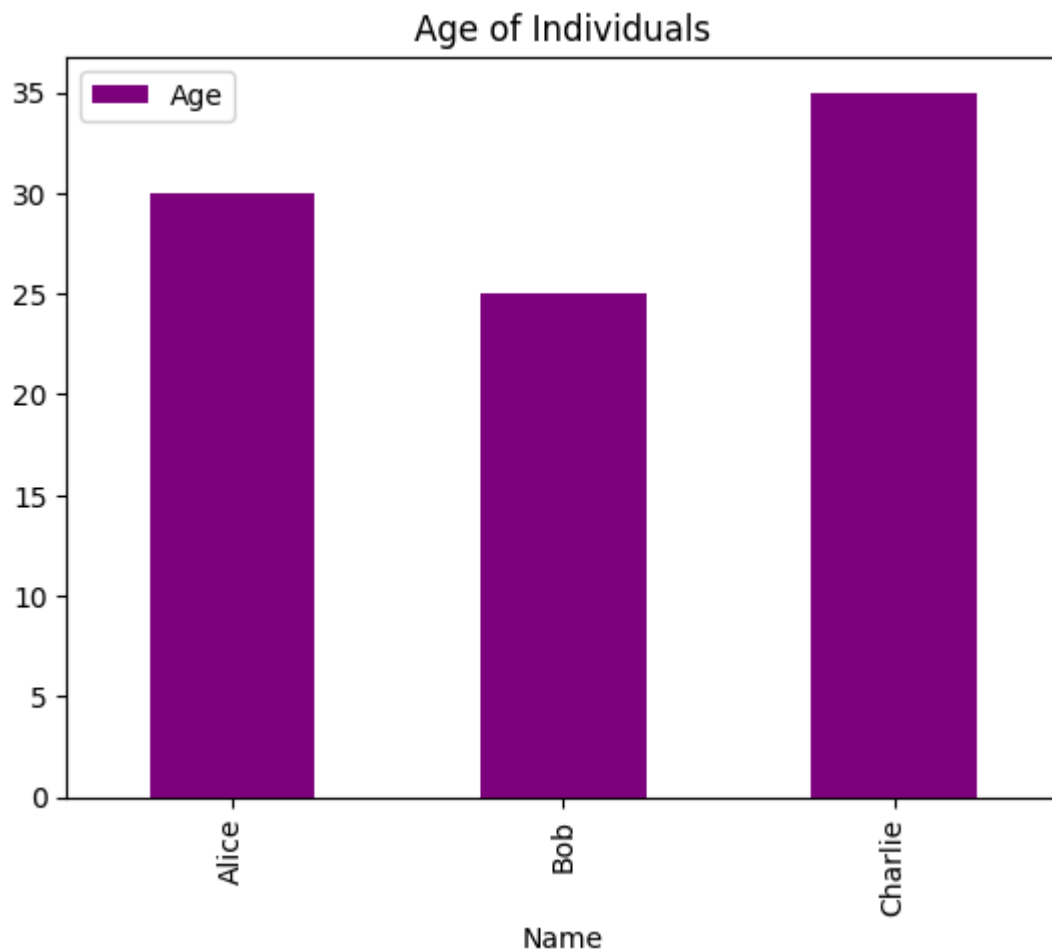


5. Matplotlib & Pandas integration

```
In [12]: data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [30, 25, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
df = pd.DataFrame(data)
print(df)

df.plot(kind='bar', x='Name', y='Age', color='purple', title='Age of Indi
plt.show()
```

	Name	Age	City
0	Alice	30	New York
1	Bob	25	Los Angeles
2	Charlie	35	Chicago



6. Advanced Customizations & Exporting

To save your chart as an image file, use `plt.savefig()` before calling `plt.show()`.

```
In [13]: x_vals = np.linspace(0, 10, 50)
y_vals = np.cos(x_vals)

plt.figure(figsize=(6, 4))
plt.plot(x_vals, y_vals, label='cos(x)', color='magenta')
plt.title('Plot Output for Production Export')
plt.xlabel('X Index')
plt.ylabel('Y Amplitude')
plt.legend()

plt.savefig('./assets/exported_chart.png', dpi=300, bbox_inches='tight')
plt.show()
```

